



Gestión de Usuarios con FastAPI, PostgreSQL, Prometheus y Grafana

Autor: Heberth Martinez

Introducción

Objetivo

El propósito de este proyecto es experimentar con la contenerización de aplicaciones mediante **Docker** y **Docker Compose**. Aquí se construirá un sistema CRUD para gestionar usuarios, con monitoreo y visualización de métricas.

Este sistema incluirá:

- Un backend **FastAPI** con operaciones CRUD sobre usuarios.
- **PostgreSQL** como base de datos.
- **Prometheus** para el monitoreo de la API.
- **Grafana** para la visualización de métricas.

⚠ Importante: Los archivos **Dockerfile** no están incluidos. Como parte del ejercicio, los estudiantes deben crearlos.

Entregables y Evaluación

- **1 punto:** Explicación y justificación del proyecto.
 - **2 puntos:** Dashboard con metricas en grafana.
 - **2 puntos:** Implementación completa con Docker y Docker Compose.
 - **Todo el código y la documentación deben estar en un repositorio Git.**
-

Guía Paso a Paso

1. Configuración del Proyecto

Crear la estructura del proyecto:

```
mkdir user-crud-monitoring && cd user-crud-monitoring
mkdir app prometheus
```

2. Implementación del Backend con FastAPI

Crear `app/main.py` :

```

from fastapi import FastAPI, HTTPException, Depends
from sqlalchemy.orm import Session
from database import SessionLocal, engine
from models import User, Base
from prometheus_client import Counter, generate_latest
from fastapi.responses import Response

app = FastAPI()

USER_CREATED = Counter("user_created", "Number of users created")

Base.metadata.create_all(bind=engine)

def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()

@app.get("/users")
def get_users(db: Session = Depends(get_db)):
    return db.query(User).all()

@app.post("/users")
def create_user(name: str, age: int, db: Session = Depends(get_db)):
    user = User(name=name, age=age)
    db.add(user)
    db.commit()
    db.refresh(user)
    USER_CREATED.inc()
    return user

@app.get("/metrics")
def metrics():
    return Response(generate_latest(), media_type="text/plain")

```

3. Configuración de la Base de Datos

Crear `app/database.py` :

```
from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker

DATABASE_URL = "postgresql://user:password@postgres:5432/mydb"
engine = create_engine(DATABASE_URL)
SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)
```

Crear `app/models.py` :

```
from sqlalchemy import Column, Integer, String
from sqlalchemy.ext.declarative import declarative_base

Base = declarative_base()

class User(Base):
    __tablename__ = "users"
    id = Column(Integer, primary_key=True, index=True)
    name = Column(String, index=True)
    age = Column(Integer)
```

4. Ejecución Manual con Docker

Nota: Los estudiantes deben escribir los **Dockerfile** para cada servicio.

Construir y ejecutar el backend:

```
docker build -t fastapi-app ./app
docker run -p 8000:8000 fastapi-app
```

Iniciar PostgreSQL de forma manual con Docker

```
docker run --name postgres_container  
-e POSTGRES_USER=user  
-e POSTGRES_PASSWORD=password  
-e POSTGRES_DB=mydb -p 5432:5432 -d postgres:13
```

Esto ejecutará un contenedor independiente de PostgreSQL con las credenciales especificadas.

5. Configuración de Prometheus

Crear `prometheus/prometheus.yml` :

```
global:  
  scrape_interval: 5s  
  
scrape_configs:  
  - job_name: "fastapi"  
    static_configs:  
      - targets: ["api:8000"]
```

Ejecutar Prometheus:

```
docker run -p 9090:9090  
-v ./prometheus/prometheus.yml:/etc/prometheus/prometheus.yml prom/prometheus
```

6. Configuración de Docker Compose

Crear `docker-compose.yml` :

```
version: "3.8"
services:
  api:
    # TODO: Agregar configuraciones necesarias
    # (variables de entorno, volúmenes, etc.)

  postgres:
    # TODO: Agregar configuraciones necesarias
    # (volúmenes, ajustes adicionales, etc.)

  prometheus:

    # TODO: Agregar configuraciones necesarias
    # (retención de datos, reglas de alertas, etc.)

  grafana:
    # TODO: Agregar configuraciones necesarias
    # (persistencia de datos, configuración de dashboards, etc.)

networks:
  backend:
```

Ejecutar todos los servicios:

```
docker-compose up --build
```

7. Pruebas de la API

Crear un usuario:

```
curl -X POST "http://localhost:8000/users?name=Alice&age=25"
```

Obtener usuarios:

```
curl -X GET "http://localhost:8000/users"
```

8. Acceso a las Métricas y Grafana

- **Prometheus UI:** <http://localhost:9090>
- **Grafana UI:** <http://localhost:3000>

En **Grafana**, agregar **Prometheus** como fuente de datos (`http://prometheus:9090`).

9. Enlaces Externos

- [Prometheus + FastAPI Integration](#)
- [Grafana Official Documentation](#)